

Sentinel AI Security Audit Report

Audit Metadata

- **Filename:** vuln.h
- **SHA-256 Checksum:** 3cd26e0d065c252824da6667ea8e979ce44742a1df48a4bb5e3cf886a6fb8fc8
- **Verdict:** VULNERABLE
- **Risk Level:** CRITICAL
- **Total Functions Evaluated:** 1

Executive Summary

The Sentinel AI automated binary analysis suite has concluded its assessment of `vuln.h`. The audit has returned a **VULNERABLE** verdict with a **CRITICAL** risk rating.

The analysis identified a high-confidence memory safety violation within the core logic of the evaluated header function. Specifically, the implementation utilizes unsafe string manipulation primitives that allow for unbounded memory writes. This flaw presents a direct path for stack-based buffer overflows, which can be leveraged by an attacker to achieve arbitrary code execution (ACE) or cause a complete system crash (DoS). Given the nature of the vulnerability, immediate remediation is required before this code is integrated into a production environment.

Analysis Metrics

- **Analysis Engine:** GATv2 (Graph Attention Network) Compiler Wrapper
- **Total Functions Scanned:** 1
- **Flagged Functions:** 1
- **Average Model Confidence:** 85.21%
- **Analysis Type:** Static Control Flow Graph (CFG) & Instruction Pattern Recognition
- **Security Standards Mapping:** CWE, SEI CERT C

Detailed Findings

Finding 1: Function `vuln\_header\_func`

- **Threat Classification:** CWE-119 / CWE-787 (Out-of-bounds Write)
- **Model Confidence:** 85.21%
- **Technical Breakdown:**

The disassembled Control Flow Graph (CFG) reveals a classic stack-based buffer overflow.

1. **Stack Allocation:** In **Basic Block 0**, the function allocates 48 bytes of stack space (`SUB RSP, 0x30`).
2. **Buffer Definition:** In **Basic Block 1**, the instruction `LEA RAX, [RBP + -0x10]` calculates the address of a local buffer starting only 16 bytes below the base pointer.

3. **Unsafe Operation:** The function then moves a user-controlled pointer (passed via ``RCX``) into ``RDX`` and calls ``strcpy`` (Address ``0x00003c40``).

4. **Exploit Vector:** Because ``strcpy`` does not perform bounds checking, providing an input string longer than 15 characters (plus the null terminator) will overwrite the saved Frame Pointer (RBP) and the Return Address on the stack. An attacker can redirect the instruction pointer to malicious shellcode or a Return-Oriented Programming (ROP) chain.

- **SEI CERT C Violation:**
- **ARR30-C:** Do not form or use out-of-bounds pointers or array subscripts.
- **STR31-C:** Guarantee that storage for strings has sufficient space for character data and the null terminator.

Remediation & Secure Code Fix

The vulnerability is caused by the use of ``strcpy``. To remediate this, the developer must use length-bounded functions and validate the size of the source string relative to the destination buffer.

Vulnerable Concept:

```
// Unsafe implementation
void vuln_header_func(char *input) {
    char buffer[16];
    if (input != NULL) {
        strcpy(buffer, input); // CRITICAL: No bounds checking
        printf("%s", buffer);
    }
}
```

Secure Implementation (Recommended):

```
#include <string.h>
#include <stdio.h>

#define MAX_BUF_SIZE 16

void vuln_header_func(const char *input) {
    char buffer[MAX_BUF_SIZE];

    if (input == NULL) {
        return;
    }

    // Use strncpy to prevent overflow, ensuring the last byte is reserved for NULL
    strncpy(buffer, input, MAX_BUF_SIZE - 1);
    buffer[MAX_BUF_SIZE - 1] = '\0'; // Explicitly null-terminate

    // Alternatively, use snprintf for safer formatting
    // snprintf(buffer, sizeof(buffer), "%s", input);

    printf("%s", buffer);
}
```

General Mitigations

To harden the binary against potential exploits targeting memory safety, the following platform-level security features and development practices should be implemented:

1. Stack Canaries (Stack Smashing Protector):

Enable compiler flags such as ``-fstack-protector-all`` (GCC/Clang) or ``/GS`` (MSVC) to insert guard values on the stack

that detect overflows before function return.

2. Address Space Layout Randomization (ASLR):

Ensure the binary is compiled as a Position Independent Executable (PIE) to randomize memory addresses, making it difficult for attackers to predict the location of the stack or linked libraries.

3. Data Execution Prevention (DEP/NX):

Mark stack and heap memory regions as non-executable to prevent the execution of injected shellcode.

4. Static & Dynamic Analysis (SAST/DAST):

Integrate automated tools into the CI/CD pipeline to catch ``strcpy``, ``gets``, and ``sprintf`` usage. Supplement this with fuzz testing (e.g., AFL++) to identify edge cases in input handling.

5. Control Flow Integrity (CFI):

Utilize hardware-backed or software-defined CFI to ensure that the program execution follows the intended Control Flow Graph, preventing unauthorized jumps to attacker-controlled memory.

End of Report

Sentinel AI Security Audit System v2.4.1